194.125 AI ML in the Era of Climate Change,Report Group 15, Assignment 1

Daniel Mazanek (12005060), Árpád Balogh (12434673), Alexander Lorenz (12239877)

February 12, 2025

1 Introduction

Assignment 1 is about post-quantization methods for Ml methods in order to tackle energy efficiennt models. ML-models have enjoyed a significant peak in interest around the globe, due to their range of potential applications. Around a decade ago, it was considered SOTA if a model could distinguish between cats and dogs; nowadays, Object detection models (ODM) perform better on such tasks than humans (even though that's mainly because it is able to distinguish between many different subspecies). Large Language Models (LLMs) like ChatGPT have reshaped entire industries, and will likely continue to do so in the future. LLMs have the potential to completely change the way humans retrieve information; at least partially, they already have.

2 Background

While the rising effectiveness of ML-models in various domains is undisputed, a big concern pops up almost simultaneously: Machine Learning models, including ODMs and especially LLMs, need a lot of energy due to their very expensive training process and also not-so-cheap inference processes. Data centers alone are estimated to consume 1/5th of the global electricity generated (mostly due to cooling), a significant part of this proportion is contributed to such model training and inference. A number that is estimated to further increase in the future, especially when chatbots become a default option in retrieving information on web browsers instead of regularly searching on the web. A search query on ChatGPT needs around 10 times more energy than a normal Google search query. There is a clear need for more efficient ML-models.

There are multiple ways to tackle this issue, such as building data centers in cold regions to reduce the energy needed for cooling. Another technique would be to decrease computational cost (and therefore increase energy efficiency) by post-quantizing models. The billions of so-called parameters in big ML-models are mostly float32-numbers which are learned during training. One can however reduce these learned parameters after the training to float16, or int8. This is of course no free lunch, as it leads to a lower precision/accuracy. The assessment, if and how much a model can be post-quantized for a given use-case, has to be made usecase-specific. The main question is therefore 'how much can we shrink the model size without tipping over a certain threshold which is defined to be the minimum precision/accuracy expected'. the quantization technique mentioned above is a so called post-training quantization. Another possible approach would be to pre-training quantization, where the model can try to learn the effect of the quantization.

One usecase where such quantization techniques are especially useful is when running such models on edge devices. This not only leads to a lower energy consumption, but can also be seen as an enabling technique to even run such models on edge devices in the first place. Running GPT-4 naive on your mobile phone would probably lead to a huge latency in response while consuming a huge portion of the battery. In order to roll out such LLMs or ODMs as planned on a larger scale for the market, quantization techniques are currently indispensable.

A fact which we will also enlighten further in a later section of this report is also the interplay between software and hardware. One reason why NVIDIA is so heavily invested in building GPUs which are optimized for float32 matrix multiplications is because that is what is mainly needed for efficiently training LLMs. However, there is also a dependency in the other direction: a theoretical improvement which is not based on float32 matrix multiplications cannot most likely not make use of the improvement with such a processing unit mentioned above.

3 Experiments

3.1 Quantization of Object Detection Models

For this part of the assignment, we have used the detr-resnet-50 model from Facebook, which is available on HuggingFace. It is based on the transformer archtitecture (encoder-decoder) with a convolutional backbone. It was trained on the COCO 2017 dataset. Furthermore, we will apply post-training quantization, which means we will change the data types of only the weights in the neural network.

For this part, we have set up a self-contained end-to-end pipeline:

1. Download COCO validation set + annotations, randomly select 1500 images
2. Download the model from huggingface
3. Post-quantize the resnet-model with TFLite to float32, float16, int8
4. Minor preprocessing to make sure all images are RGB-encoded
5. Run inference on all randomly selected images with all 4 models
6. Run COCOeval code to obtain average precision, average recall
7. Store evaluation, inference time and memory for all 4 models in a json-file

For the steps 1 and 2, we have implemented a data loader that downloads neccessary annotations for evaluation, as well as the ssd-mobilenet model for quantization. From approximately 40k downloaded images, we will only consider 1,500 random images for evaluation that we load in our environment. In the following, the ssd-mobilenet model is loaded and converted to tflite, a format that allows models to run on edge devices. Next, we specify the optimization strategy that we want to apply. In our case we assign the converter to use 32-bit floating point and 16-bit floating point. By using the defaults for *converter.optimizations* parameter, the converter transforms the weights to 8-bit integers. Consequently, we end up with 4 different models that we want to inspect in their behavior.

After that, we set up a loop that for each model firstly obtains the predictions on the loaded set of images, secondly store the results, and lastly evaluates the performance metrices such as average precision, average recall, mean inference time and memory footprint.

Before inferencing, the input image is transformed to the correct format for our model, meaning the image is first converted to RGB, then transformed to an image tensor with pixel normalized between 0 and 1, and then scaled back to the range 0-255 followed by an added batch dimension. Post inferencing, we construct a result json in a defined format that is required for evaluation [1] using the evaluation code of *COCOeval* module [2], which allows us to obtain predictions metrics i.e. average precision and average recall for each model.

```
[{
    "image_id": int,
    "category_id": int,
    "bbox": [x,y,width,height],
    "score": float
}]
```

Listing 1: Required JSON structure for COCOeval evaluation code

[1] https://cocodataset.org/format-results
[2] https://cocodataset.org/detection-eval

Finally, we store the acquired performances metrics, mean inference time, and memory footprint for further analysis.

3.2 Quantization of Large Language Models (LLMs)

For this part of the assignment, we chose to work with the GPT2 model by OpenAI, from HuggingFace. The model was trained on raw English text, which contained zero human labeling. For this reason the model is best at generating texts from prompts, however it has the ability to guess the next word in sentences.

For this part, we tested the guessing ability of the model on the Lambada dataset:

1. Download the LAMBADA dataset and select a random subset of it
2. Download the transformers python package and import the GPT2 model
3. Quantize the model with OpenGPT to int8, int4 and int2
4. Implement an evaluation function, where the model needs to guess the last word of a sentence
5. Evaluate the accuracy and speed of the model and store the observations

For parts 1 and 2 we ensured that all necessary libraries were installed. Before the quantization began, we wanted to test the regular model, to make sure everything is set up and working as intended in the local environment. Next we discussed what the optimal sample size of the dataset should be, since the dataset is enormous. We decided, that 1000 random samples from the dataset should be satisfactory to give us a picture of the accuracy and speed of the different models. For the accuracy model, we tried out 3 different top-k values to see how much improvement it makes. First we simply tested top-1 accuracy, to see how well the model does on its initial guesses. After that, we added top-5 and top-10 accuracies to the research. We thought that if the guess is in the first 5 predictions, it is still acceptable, since the dataset uses a high level of English and contains complicated sentences. With the top-10 accuracy we were curious if the accuracy increases enough to justify the considerably higher computational time.

After the initial test on the regular GPT2 model, we began the quantization. As specified by the assignment, we converted the weights of the model from 32-bit floating point to 8-bit, 4-bit and 2-bit integers as well. We ended up with 4 models in this case as well, so we can analyze and fully understand how quantization affects the overall performance of the model.

The evaluation of the model is based on the context of the given sample and the last word. We decided that the token should be every word apart from the last in a sample. Making it bigger would significantly increase computational time and cost. On the other side, having it smaller would give us close to no accurate predictions. After setting everything up, we started a timer (using Python's time library) so we could measure how many tokens per second were computed. Then we generated outputs for each sample and checked if the target word is the prediction (top-1 accuracy) or is within the generated words (top-5 ad top-10 accuracy).

Finally we stored the measured data (accuracy, speed and model size) for further analysis.

4 Results

4.1 Quantization of Object Detection Models

The results presented in this section are obtained using an Intel 11th Generation CPU. The performance metrics, namely Average Precision (AP) and Average Recall (AR), were calculated based on predictions made on 1,500 images.

4.1.1 Model Comparison

At first, we are interested in on how the post-train quantization effected the actual model size, as well as its mean inference time. Therefore, we have created the following to summarize the results.

Table 1: Model Comparison

Model	Memory Footprint	Mean Inference Time
ssd_mobilenet_float32	23.720039	0.115962
ssd_mobilenet_float16	12.166817	0.117698
ssd_mobilenet_int8	6.706703	0.167178

The float32 model has the largest memory footprint of 23.72 MB, while preserving the fastest mean average inference time with 0.1160s. Moving to the float16 model, it reduces memory usage by almost half (12.17 MB) while maintaining nearly the same inference time (0.1177s). Finally, the int8 model further reduces memory by factor of 3.5 to just 6.71 MB, but the inference time increases to 0.1672s.

In terms of reduction of model size, these results align with the proposed benefits from Google [3].

However, even though floating points models preserve higher precision, that eventually leads to increased computational complexity and model size, both models obtain faster inference time compared to the 8-Bit weighted model. These observations are in contrast to the suggested speedup of 2 to 3 times faster execution on a CPU [4] .

This could be due to reason that modern hardware (i.e. CPUs and GPUs) may be optimized for computing more precise numerical representation.

This demonstrates in our opinion that while such quantized models can drastically reduce the memory needed, it is essential to also have the proper hardware (i.e., processing units which actually can make use of float16 or int8 directly and not convert them back to float32).

4.1.2 Prediction Accuracy

In the following, we evaluate the average precision and recall for each model performance across small, medium, large, and all object sizes. The evaluation is based on predicted bounding boxes that match the ground truth annotations with Intersection over Union (IoU) thresholds ranging from 0.5 to 0.95.

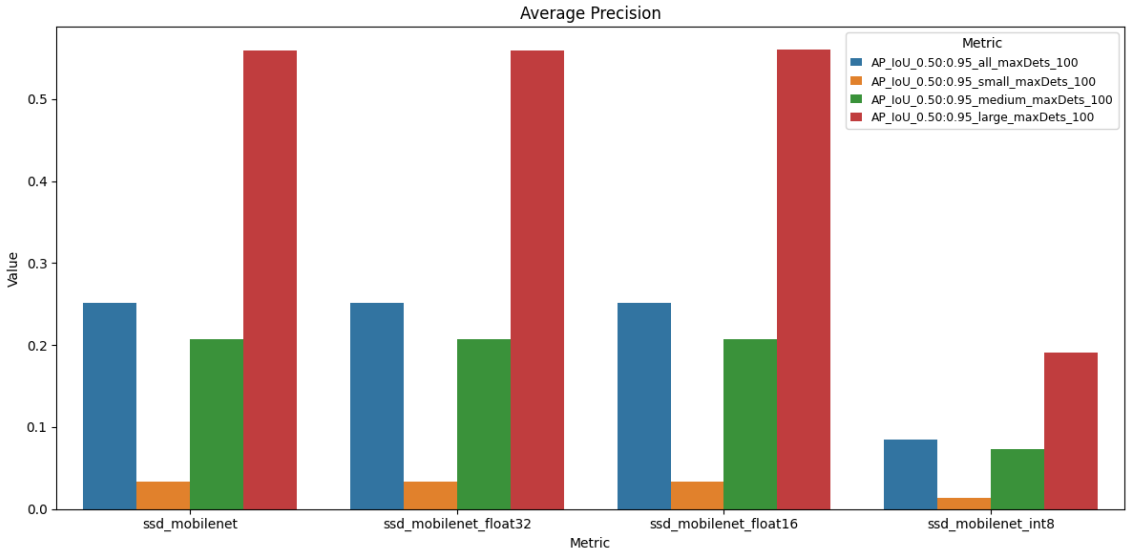

Figure 1: Model Performance Comparison - Average Precision (AP)

The bar plot illustrates the average precision for each model. In general precision measure, the correctness of the labeling, reflecting how many of the predicted positive cases were actually correct.

Overall, it appears that the model struggles to accurately predict small and medium-sized objects. However, for larger objects, it assigns the correct class to the objects with a precision of approximately 55%

[3] https://ai.google.dev/edge/litert/models/post$_t raining_q uantization$
[4] https://ai.google.dev/edge/litert/models/post$_t raining_q uantization$

Furthermore, the plot highlights that the precision remains consistent between the float32 and float16 models, while the int8 model experiences a noticeable drop in precision. At this point, the int8 model shows for all image sizes overall unsatisfactory performance.

In the following plot, we inspect the average recall metrics of each model.

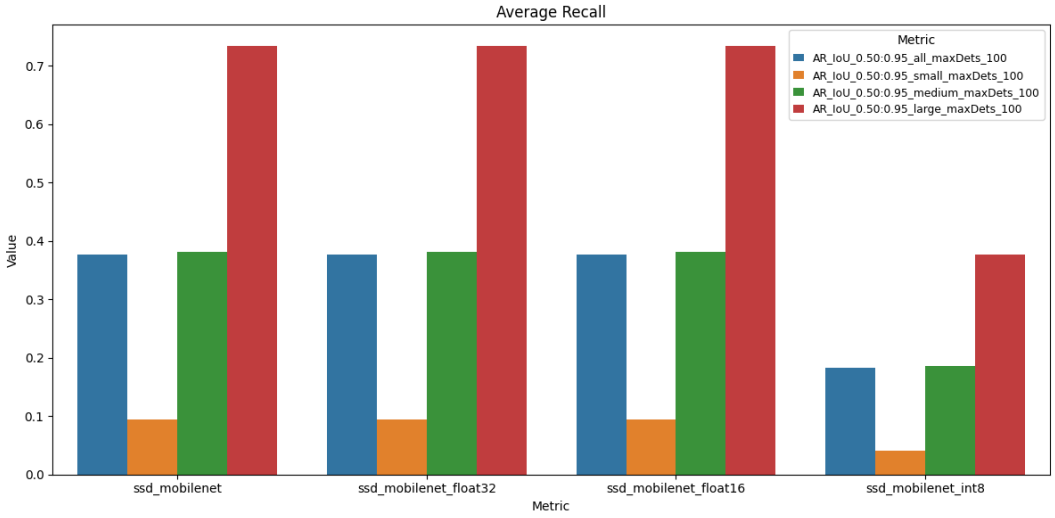

Figure 2: Model Performance Comparison - Average Recall (AP)

In general, recall measures the completeness of the model's predictions, indicating the proportion of actual true classes that were correctly identified as positive.

Similarly to the results from the section before, the recall reveals that models tends to make more accurate predictions for large objects.

However, the 8-bit model reveals the poorest performance.

4.2 Quantization of Large Language Models (LLMs)

The following results are obtained on a machine equipped with a 9th generation AMD CPU. The performance metric, namely Top-k Accuracy and Tokens per Second, were calculated based on 1000 random samples taken from the LAMBADA dataset.

4.2.1 Model Comparison

Firstly, we were interested to see the quantization affected the model size. The following table shows our measurements:

Table 2: Model Comparison

Model	Size in memory(MB)
GTP2	474.72
GPT2-int8	118.68
GPT2-int4	59.34
GPT2-int2	29.67

Obviously, the standard GPT2 model has the largest memory footprint of 474.72 MB. The int8 model reduces this size to less than a quarter of the standard. The int4 and int2 models are around half of the previous ones, which is to be expected.

4.2.2 Prediction accuracy

In this section, we evaluate the accuracy and speed of the models across the top-1, top-5, top-10 accuracy cases. The evaluation is based on the prediction of the last word in the samples.

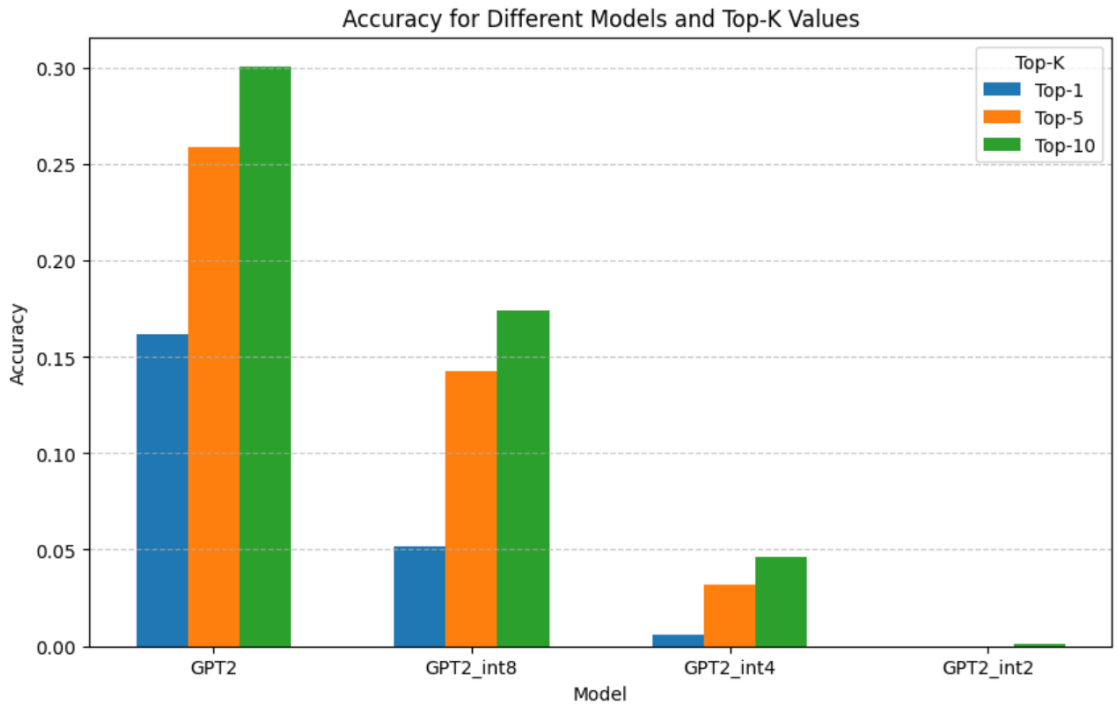

Figure 3: Model Performance Comparison - Accuracy

The bar plot shows the prediction ability of each model. Overall, it seems the model is struggling with the prediction on its first try, even the standard model can only predict 15% of the words correctly. This drastically declines, with the int8 model only achieving 0.05 and the int4 model less than 0.01. The int2 model did not get any predictions correct, not even in its first five guesses. This is to be expected, since the dataset is extremely difficult, intended for the highest quality LLMs.

The top-5 accuracy shows significant improvement in each model, except the int2. For the regular and int8 models, the improvement is around 0.1, which is considerable. The int4 model's accuracy rises to around 0.03, which is 6 times better than its previous.

The top-10 accuracy also shows improvements, but not as notable as the top-5 showed. The major takeaway here is that we could increase this to top-20 even top-100 if we wanted, but the improvements are going to be less and less, since the model will start generating the same words, even gibberish the more words it is required to predict. Even adding a penalty for repetition did not stop the model from generating the same words twice or three times for the top-10 measurements. The int2 model actually got a prediction right in this case, which we think is complete luck, since it generated the same words for almost every sample we tried it with.

4.2.3 Speed comparison

In this section, we analyzed how the quantization affects the model's speed. The metric used for this is tokens per second, which represents how many tokens the model can process in a second.

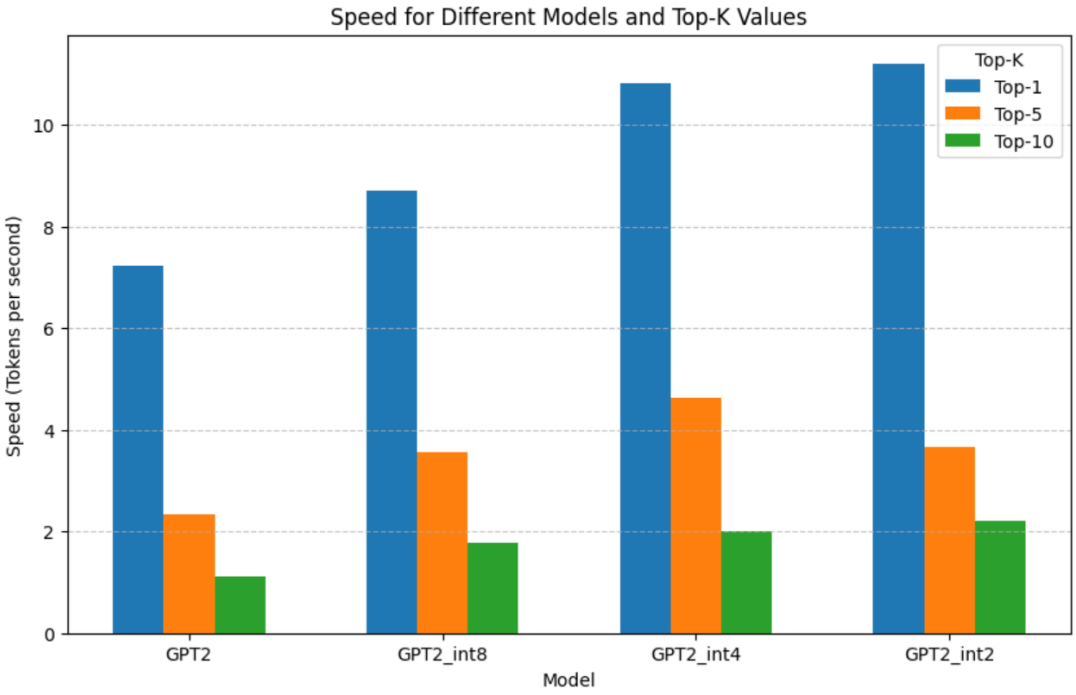

Figure 4: Model Performance Comparison - Speed

The graph above shows the speed of the different models for the different k values. We though it would be interesting to analyze how much time we have to sacrifice in order to have a wider range of guesses by the model. Firstly, the standard model is pretty fast, with a speed ca. 7 tokens per second, meaning it finishes giving predictions for the 1000 random samples in 2 minutes. However it is visible that generating five predictions takes significantly longer for the model. Considering the 5 times increase in generation, it is not unexpected. Generating 10 words is obviously the slowest, but not as slow as we initially thought it would be.

Moving on to the int8 model, we were surprised to see that the improvement in speed is underwhelming. Sacrificing almost half of the accuracy for only a 20% increase in speed was unexpected at first, because in theory it should double the speed of the model.

Even more surprising were the results for the int4 and int2 models. On paper these should provide a 4 and 16 times speed increase respectively, however in practice they were almost identical. We later learned that these quantization methods have no native hardware support on most machines and CPUs, so the extra overhead causes the models to not reach their full potential.

5 Conclusions

In the case of quantizing an object detection model, we gained more insights into the techniques and steps involved in post-training quantization. Unexpectedly, both the float16 model, and especially the int8 model, resulted in slower inference times. Given the reduced bit representation, we had expected fewer bits per computation to lead to faster inference. However, our results show the opposite. It appears that GPUs are particularly optimized for float32 operations, while CPUs must convert the numerical representation back to float32, leading to longer inference times.

Undoubtedly, we observed that performing quantization to float16 significantly reduces the model size by half, with only minimal loss in prediction accuracy. However, the performance with 8-bit weight representation was quite poor. Nevertheless, techniques such as quantization-aware training can be employed to improve accuracy in such cases.

About the quantization of large language models we learned about the different effects it can have on the model performance. As we expected, the original model was the slowest, yet most accurate. On the other hand we did not expect the scale of the differences between different methods. The int4 and

int2 models severely underperformed our expectations and in our research they seem almost useless for any meaningful observations.

Naturally, there are aspects of this experiment we could improve in the future, such as choosing a better model and conducting the research on a well-equipped machine with a strong GPU, ideally optimized for various different numerical data types. However this assignment helped us understand how the models operate, how does quantization effect the performance and gave us insights on the more practical usage of artificial intelligence.

Last but not least, the fact that hardware also plays an important role in actually being able to make use of the accelerations leads to the insight that post-quantization is not an encapsulated method for increased efficiency, but part of a bigger overall strategy. This strategy has to include other parties such as ones from the hardware domain. If and how this can be solved efficiently remains an open question. As Alan Kay said: 'People who are really serious about software should make their own hardware'.